

Proyecto - P3

Implementación de un juego basado en estrategia por turnos

Objetivo

Hasta ahora el juego se ha ido ampliando en base a un diseño inicial bastante simple. A medida que se hacen extensiones de un proyecto llega siempre el momento en que hay que replantearse reestructuraciones para que pueda seguir evolucionando de manera sencilla en el futuro. Cambios tan simples como exigir que cuando al imprimir mensajes por la consola se corten al llegar a un máximo de caracteres, o que en lugar de imprimir por consola se haga en una interfaz gráfica pueden eternizarse si el diseño inicial no ha identificado como encapsular esta funcionalidad.

El objetivo de esta práctica es incidir en el uso de técnicas orientadas a objetos como encapsulación, herencia y polimorfismo para mejorar la extensibilidad de la aplicación y aprender a variar el comportamiento del juego sin necesidad de cambiar drásticamente todo el código.

Evaluación de la 3ª entrega del proyecto

Los requisitos que se evaluarán serán los siguientes:

	Funcionalidades generales del juego	Puntuación (6)
30	Pedir la clase a la que puede pertenecer el jugador al inicio, aparte del nombre. Se deben soportar al menos tres clases, por ejemplo Marine, <i>Francotirador</i> , <i>Zapador</i> . <i>Ver tarea 37.</i>	1
31	Seleccionar al inicio si cargar el juego por defecto (cualquier juego muy sencillo con al menos 36 celdas, 30% de las celdas con objetos, al menos 2 enemigos de distinto tipo), o cargar juego a partir de un conjunto de ficheros (lo que se hizo en la segunda parte del proyecto). <i>Ver ejemplo 1 y tarea 43.</i>	2
32	El bucle principal del juego debe ser responsable de capturar las excepciones que ocurran y procesarlas (imprimir mensajes de error si hiciese falta). <i>Ver ejemplo 2.</i>	1.5
33	Soportar el uso de comandos compuestos para los movimientos a través de la consola. Ejemplo "atacar 4e 2" o "atacar 4e alien_azul 2" para atacar 2 veces a todos los enemigos de la celda 4e o al alien azul, respectivamente.	1.5

	Ver ejemplo 3 y tareas 40, 42.	
--	--------------------------------	--

	Diseño del juego	Puntuación (48)
34	Crear una clase Juego. La clase juego debe tener acceso al jugador, al mapa, a las celdas, y a todos los componentes que estén relacionados con juego, que dependerá de la implementación de cada uno. Esta clase servirá para que el resto de tareas de esta práctica se resuelvan de una manera sencilla (por ejemplo, al cargar un juego, que incluye el mapa, los enemigos, etc).	1
35	Jerarquía de personajes. Los personajes deberán estar jerarquizados. Habrá una clase raíz llamada Personaje, que defina los métodos y atributos comunes; un segundo nivel compuesto por una clase Jugador y una clase Enemigo que extienden de Personaje. En el tercer nivel habrá tres clases que representan los tipos de Jugador, por ejemplo Marine, Francotirador, y Zapador, y dos tipos de Enemigo (en este caso aliens), Sectoid, y Floater, y dos subtipos de Floater: LightFloater y HeavyFloater. La clase raíz Personaje, debe tener al menos los siguientes métodos: - Mover(dirección) - Coger(Objeto objeto) - Atacar(Personaje personaje) - Tirar(Objeto objeto) - Equipar(Arma o Armadura) - Desequipar(Arma o Armadura) Además, la clase Personaje no puede tener un constructor vacío sin argumentos. Deberá contar con uno o varios constructores que acepten los atributos obligatorios de un personaje, como mínimo el nombre del personaje. Cada clase tendrá que sobrecribir los métodos para establecer diferentes comportamientos (polimorfismo). Al menos los siguientes métodos tienen que comportarse de forma distinta: - Atacar: Los marines hacen el doble de daño a corta distancia y los francotiradores hacen el $(\text{casillas de separación})^{1.2}$ más de daño a larga distancia. Adicionalmente, tanto el zapador como el marine, al tener mala puntería, sólo harán un 5% de daño cuando ataquen con un arma a más de 2 casillas de distancia. - Mover: La energía consumida por una clase u otra serán distintas en función de la clase (ej. marine es más pesado y consume más energía al moverse). - Equipar. El marine tiene la habilidad especial de poder equipar dos armas a dos manos simultáneamente. En ese caso, su consumo de energía aumentará 1.5 veces la cantidad de energía consumida. - Coger. Solo el zapador estará capacitado para cargar explosivos. - Los métodos que sean iguales para distintos tipos de personajes deben estar implementados en la clase Personaje.	9

	Por último hay que tener en cuenta que en jerarquías de clases es habitual que no todos los atributos sean privados. Muchos de ellos pueden tener otro nivel de acceso, como <i>protected</i>, ya que las clases de una jerarquía no son más que una extensión del modelo, por lo que no se rompe la encapsulación. No obstante habrá que justificar correctamente las decisiones tomadas.	
36	Jerarquía de objetos. Los objetos deberán estar jerarquizados. Habrá una clase raíz llamada Objeto. Esta clase, además de los atributos nombre y descripción, que son comunes a todos los objetos, deberá tener el método usar. Este método al menos deberá recibir una instancia del personaje que lo usa: <i>void usar(Personaje personaje)</i>. Cada tipo de objeto deberá implementar esto de manera acorde. Al menos habrá los siguientes tipos de objetos: - botiquin: su método usar suma a la vida del jugador, pero no puede exceder su vida máxima. - arma: su método usar lanza una excepción de no usabilidad. - armadura: su método usar lanza una excepción de no usabilidad. - binocular: su método usar permite incrementar el rango de visión durante el turno actual. En caso de tener un binocular ya equipado, se mostrará el de mayor rango. - torito rojo: su método usar incrementará la energía del jugador en ese turno, pero nunca podrá exceder de su máxima energía. Un efecto colateral derivado de la toritina es que en el siguiente turno, el jugador tendrá un 10% menos de energía.	8
37	Jerarquía de excepciones. Los errores derivados de las acciones del usuario deberán ser controlados con excepciones. En el momento en el que se detecte una condición que impida realizar una acción (por ejemplo, la casilla a donde se quiere mover al personaje no es transitable, el objeto que se quiere coger no existe, etc) se lanzará una excepción que será capturada en el bucle principal para poder indicarle el error al usuario. Todo tipo de excepción es aquel que extiende la clase <i>Exception</i> de Java y se deberán definir al menos dos tipos propios de excepciones que deberán ser tratados de forma diferente desde el bucle principal.	6
38	Interfaz Comando. Los comandos serán representados por una interfaz Comando. Esta interfaz tendrá un método <i>void ejecutar()</i> que no recibe argumentos ni devuelve nada, pero que lleva a cabo la acción que sea dependiendo de la clase que implemente la interfaz Comando. El método <i>ejecutar()</i> además declara la excepción <i>ComandoExcepcion</i> que puede ser lanzada si hay algún error procesando el comando. Los comandos que tendrán que implementar la interfaz serán los correspondientes con las acciones del usuario: <i>mover</i>, <i>coger</i>, <i>atacar</i>, <i>usar</i>, etc. Cada clase por tanto será responsable de verificar que se puede ejecutar el comando, invocar los métodos correspondientes para realizar el comando, y en caso de que haya un error lanzar el <i>ComandoExcepcion</i>. <i>Ver ejemplo 3.</i>	4
39	Interfaz Consola. Esta interfaz debe declarar la funcionalidad de imprimir mensajes y pedir datos, de manera que NO puede haber llamadas a lo largo del código a <i>System.out.println</i>. Las llamadas deberían ser a	5

	<i>consola.imprimir(String mensaje)</i>. De la misma forma, cada vez que se quiera preguntar al usuario por un dato, se deberá usar el método de <i>public String consola.leer(String descripcion)</i>, donde descripción es el mensaje que se le dice al usuario antes de esperar a que introduzca los datos. Por ejemplo <i>consola.leer("Introduce nombre:")</i> debería imprimir por pantalla "Introduce nombre:", leer con <i>Scanner</i> (por ejemplo) lo que meta el usuario y devolver los datos introducidos como un <i>String</i>. Se proporcionará al menos una implementación de esta interfaz, <i>ConsolaNormal</i>, que imprimirá usando <i>System.out</i> y leerá usando por ejemplo la clase <i>Scanner</i> de Java (o la que se prefiera). Usar esta interfaz en el juego hace que sea muy fácil decidir nuevas estrategias para representar los datos. Por ejemplo, se podría pensar en la implementación <i>ConsolaGrafica</i> que en lugar de imprimir por consola lo haga en una interfaz gráfica, o una <i>ConsolaMultilingue</i> que permita traducir los mensajes, todo de manera transparente para el resto del juego.	
40	Comandos compuestos. Se podrán construir comandos compuestos a partir de comandos simples. Los comandos compuestos pueden estar formado tanto por comandos simples como por otros compuestos (un árbol de comandos). Esto es útil para definir tareas complicadas como "atacar 4e alien_azul 2" para intentar atacar al alien azul dos veces o "equipar lanzacohetes ametralladora" que indica que el usuario quiere equipar el lanzacohetes y desequipar la ametralladora. Para implementar esta parte, es necesario hacer uso de la interfaz <i>Comando</i>. Habrá que implementar además los comandos concretos: <i>ComandoMover</i>, <i>ComandoMirar</i>, <i>ComandoCoger</i>... correspondientes a las acciones de las anteriores entregas. Habrá también una clase <i>ComandoCompuesto</i> (que implementa <i>Comando</i>) que tendrá una lista de comandos que maneja. El método <i>ejecutar()</i> de comando compuesto deberá recorrer la lista de comandos que contiene e invocar el método <i>ejecutar</i> de cada uno. Nótese que, dado que <i>ComandoCompuesto</i> debe tener una lista de comandos (interfaz <i>Comando</i>), éstos podrán ser tanto simples (<i>ComandoMover</i>, <i>ComandoMirar</i>...) como compuestos (cualquiera de tipo <i>ComandoCompuesto</i>). Habrá que crear también un tipo de comando compuesto, <i>ComandoRepetido</i>. La intención de este comando es la de poder ejecutar <i>N</i> veces un comando cualquiera (interfaz <i>Comando</i>), que puede ser tanto simple como compuesto. Para implementar esta parte, puede ser de ayuda esta referencia.	10
41	Interfaz CargadorJuego. Esta interfaz define un método para poder cargar un juego llamado <i>cargarJuego()</i> sin parámetros que devuelve una instancia de <i>Juego</i>. La clase principal deberá hacer uso de esta interfaz para inicializar el juego al principio del programa. La estrategia de inicialización dependerá de la clase que implemente dicha interfaz. Habrá dos implementaciones: - CargadorJuegoPorDefecto. Esta clase lo único que hace la implementación del método <i>cargarJuego()</i> es crear un personaje,	5

	el mapa (tal y como se hacía en la primera entrega), las celdas de mapa y en definitiva todo lo necesario para poder tener un juego al que jugar. - <code>CargadorJuegoDeFicheros</code>. EL método <code>cargarJuego()</code> de esta clase lo que debe hacer es leer los ficheros y construir el juego, exactamente igual que se hacía en la práctica 2. En el <i>ejemplo 1</i> se puede ver el uso del <code>CargadorJuego</code> desde la clase principal. Es posible que <code>CargadorJuegoDeFicheros</code> necesite distintos parámetros en el constructor.	
--	---	--

	Requisitos adicionales	Puntuación (6)
42	Correcto uso de modificadores de clase como <code>abstract</code> o <code>final</code> en las jerarquías. Los modificadores anteriores deben usarse tanto en la jerarquía de personajes como de objetos. Justificar su uso.	2
43	Implementar algún método abstracto consistente con la definición de la jerarquía. El método abstracto debe claramente definir una funcionalidad común a las clases hijas pero con un comportamiento claramente diferenciado.	1
44	Utilizar la implementación del método de la clase padre en alguno de los métodos sobrescritos. El uso del método de la clase padre ha de tener una clara justificación.	1
45	Uso de la instrucción <code>instanceof</code> en el método <code>coger del personaje para definir comportamientos distintos en función del tipo de objeto</code>	1
46	Declaración de una o varias constantes en el programa. Las constantes han de tener una clara justificación. Como buena práctica, las constantes deben ir definidas en una clase <i>final</i> (no se puede extender) con constructor privado (no se puede instanciar), y los atributos deben ser <i>public static final</i> (se pueden acceder a ellos desde fuera sin necesidad de instanciar la clase y no se pueden modificar).	1

Anexo de ejemplos

Ejemplo 1

```
CargadorJuego cargador = null;

if (cargarDeFicheros) {
    cargador = new CargadorJuegoDeFicheros();
} else {
    cargador = new CargadorJuegoPorDefecto();
}

Juego juego = cargador.cargarJuego();
...
```

Ejemplo 2

```
Scanner scanner = new Scanner(System.in);
while(true) {
    final String comando = scanner.nextLine();
    try {
        if ("mover".equals(comando)) {
            ...
        } else if ...
    } catch (ComandoException ex) {
        // Hacer algo con la excepción. Pj: imprimir el mensaje de error por pantalla
        ...
    }
}
```

Ejemplo 3

```
while(true) {
    final String comando = scanner.nextLine();
    try {
        if (...) {
            // Atacar posición 4e
            new ComandoAtacar("4d", personaje).ejecutar();
        } else if (...) {
            new ComandoRepetido(new ComandoAtacar("4d", personaje), 2).ejecutar();
        }
    } catch (ComandoException e) {
        // Hacer algo con la excepción. Pj: imprimir el mensaje de error por pantalla.
        ...
    }
}
```

